بسم الله الرحمن الرحيم

Structural Honesty Verification as a Complementary Static Analysis Class

An Empirical Benchmark of furqan-lint Across 25 Open-Source Python Codebases

Bilal Syed Arfeen
BayyinahEnterprise
November 2026

Abstract

We present an empirical evaluation of furqan-lint v2.1.0, a multi-language static analysis tool that checks for *structural honesty* -- whether a program's surface behavior (what it claims to do) matches its substrate behavior (what it actually does). We benchmark furqan-lint against bandit across 25 widely-used open-source Python packages totaling 2,327,868 lines of code and 5,252 files. furqan-lint produces more findings than bandit in 22 of 25 packages (median per-package ratio 5.0x; pooled ratio 0.83x, pulled below 1.0 by three scientific computing packages where bandit's assert-usage rule dominates). Two rules -- SI-001 (Any type annotation) and SI-006 (assert in non-test code) -- account for 58.6% of furqan-lint's output. We argue that structural honesty verification occupies a region of the analysis space that existing linters leave unexamined, but we do not decompose the overlap between tools in this study. We provide the full dataset, including raw JSON output from both tools, for reproducibility.

Note on terminology. Throughout this paper, finding means a pattern flagged by a tool's rule. Some findings are true positives; others are intentional design choices. The false positive rate is not measured. We present the results as evidence that structural honesty verification produces different counts from different rules, not as a claim that every finding represents a defect.

---

1. Introduction

Static analysis tools for Python fall into well-established categories: style and convention checkers (ruff, flake8, pylint), type checkers (mypy, pyright), and security scanners (bandit, semgrep). Each category targets a specific finding class: formatting inconsistencies, type errors, and known vulnerability patterns, respectively.

We identify a fourth class that existing tools do not systematically address: *structural dishonesty* -- code whose observable interface makes claims that its implementation does not honor. Examples include:

- An **except Exception** block that catches everything but re-raises nothing.
- A parameter named **password** assigned the literal string "admin123".
- ****kwargs** accepted and passed through without validation.

furqan-lint v2.1.0 implements SHV as 42 Python rules in four families:

- **RING-CLOSE (RC)**: resources that open but do not close (5 rules)
- **MARAD (MR)**: failures that produce no signal (5 rules)

- **TRUST-BOUNDARY (TB)**: security boundaries that can be crossed (25 active + 3 informational, off by default)
- **SCAN-INCOMPLETE (SI)**: regions where analysis is explicitly abandoned (7 rules)

Three rules (TB-001/002/003) are informational and disabled by default. All results use the default: 39 active Python rules. The tool additionally implements 10 adapter-only rules for C, C++, Java, C#, and Ada/SPARK (52 rule IDs total across six languages).

2. Method

2.1 Package selection

We selected 25 packages from PyPI spanning web frameworks, HTTP clients, data science, ORMs, task queues, templating, cryptography, SSH, scraping, CLIs, cloud SDKs, and image processing. Size ranges from 6,385 lines (requests) to 635,499 (pandas). No package was excluded based on preliminary results.

2.2 Tools compared

Tool	Version	Configuration
furqan-lint	v2.1.0 (131ab8b)	Default (39 active Python rules)
bandit	1.9.4	Default (all rules, no exclusions)

2.3 Procedure

Each package was scanned in its installed form from PyPI (site-packages). Both tools were run with default configuration.

Asymmetry note. furqan-lint's SI-006 (assert usage) exempts test files. bandit's B101 does not. Installed packages include test files (~36% of total lines). This asymmetry is the primary driver of the 0.2x--0.3x ratios in numpy, pandas, and scipy.

2.4 Threats to validity

1. Overlap. We do not decompose findings into overlapping and unique categories.

2. Precision. Neither tool's false positive rate is measured. SI-001 (47.1% of output) may have a high false-positive rate in gradually-typed codebases.

3. Test-file asymmetry. SI-006 and B101 treat test files differently.

4. Rule concentration. Two rules produce 58.6% of output.

5. SI-001 overlaps with ruff ANN401. The largest rule detects a pattern an existing linter already flags.

3. Results

3.1 Aggregate findings

Metric	Value
Packages scanned	25
Total Python files	5,252
Total lines of code	2,327,868
Total furqan-lint findings	24,933
Total bandit findings	29,959
Pooled ratio (furqan / bandit)	0.83x
Packages where furqan > bandit	22 of 25

Metric	Value
Packages where bandit > furqan	3 of 25

3.2 Per-package results

Package	Domain	Files	Lines	furqan	bandit	Ratio
django	Web	909	165,634	1,605	281	5.7x
flask	Micro web	24	9,199	76	13	5.8x
fastapi	Async web	52	22,748	688	70	9.8x
tornado	Async net.	76	50,586	947	285	3.3x
starlette	ASGI	36	6,947	313	73	4.3x
requests	HTTP	19	6,385	149	16	9.3x
httpx	Async HTTP	23	8,828	84	29	2.9x
urllib3	HTTP low	36	11,914	99	10	9.9x
aiohttp	Async HTTP	54	28,467	662	178	3.7x
sqlalchemy	SQL ORM	258	253,997	8,496	866	9.8x
celery	Task queue	161	47,496	703	49	14.3x
jinja2	Template	25	14,425	239	62	3.9x
pydantic	Validation	105	45,762	2,099	86	24.4x
cryptography	Crypto	78	13,868	52	18	2.9x
paramiko	SSH	42	18,621	204	21	9.7x
scrapy	Scraping	199	38,503	1,361	241	5.6x
click	CLI	17	12,674	94	36	2.6x
rich	Terminal	100	38,515	200	58	3.4x
typer	CLI builder	32	13,911	486	60	8.1x
boto3	AWS SDK	39	8,558	38	0	--
PIL	Image	98	35,149	652	288	2.3x
werkzeug	WSGI	52	21,687	184	28	6.6x
numpy	Numerical	407	243,710	1,018	3,597	0.3x
pandas	Data	1,422	635,499	2,657	17,701	0.2x
scipy	Scientific	988	574,785	1,827	5,893	0.3x

3.3 Per-package ratio distribution

Metric	Value
Mean (n=24, excl. boto3)	6.2x
Median	5.0x
Std dev	5.5
Min	0.2x (pandas)
Max	24.4x (pydantic)

3.4 Severity distribution

Severity	Count	Percentage
CRITICAL	331	1.3%
HIGH	1,918	7.7%

Severity	Count	Percentage
MEDIUM	17,800	71.4%
LOW	4,884	19.6%
Total	24,933	100%

CRITICAL findings include TB-006 (pickle/yaml deserialization, 153), TB-005 (eval/exec, 115), MR-001 (bare except, 29), and TB-029 (C buffer overflow in numpy's bundled C code, 34). The TB-029 findings come from tree-sitter scanning .c files, not the 39 Python rules.

3.5 Per-rule concentration

Rule	Count	% total	Description
SI-001	11,742	47.1%	Any type annotation
SI-006	2,872	11.5%	assert in non-test code
SI-005	2,794	11.2%	**kwargs passthrough
SI-007	2,017	8.1%	hasattr() swallows AttributeError
SI-004	1,894	7.6%	TODO/FIXME comment
MR-005	964	3.9%	Empty except block
Other (35)	2,650	10.6%	

Two rules (SI-001 and SI-006) produce 58.6% of all findings. The top 6 rules produce 89.4%.

4. Discussion

4.1 Rule concentration

SI-001 (Any annotation, 47.1%) fires on every use of typing.Any. ruff's ANN401 detects the same pattern, so SI-001 findings are not unique to structural honesty analysis. Whether a given Any is a genuine structural gap or an acceptable transition depends on the project's typing maturity.

SI-006 (assert, 11.5%) fires on assert outside test paths. This overlaps with bandit's B101, though with different test-file treatment. The remaining rules each contribute under 12%.

4.2 Why the ratio varies

High-ratio packages (pydantic 24.4x, celery 14.3x): Few bandit findings but many SI-001/SI-005/SI-007 hits from furqan-lint.

Low-ratio packages (pandas 0.2x, numpy 0.3x, scipy 0.3x): bandit's B101 fires on test assertions that SI-006 skips. This asymmetry, not a difference in detection capability, drives the reversal.

4.3 Complementarity

Many furqan-lint rules have no bandit equivalent (error handling analysis, resource lifecycle, kwargs passthrough). Conversely, bandit checks import-level patterns that furqan-lint omits. Whether the additional findings are truly unique requires an overlap decomposition we have not performed.

4.4 Limitations

1. Finding count is not defect count. SI-001 (47.1%) may have a high false-positive rate.

2. No overlap decomposition.

3. Rule concentration: two rules produce 58.6% of output.

4. Test-file asymmetry between SI-006 and B101.

5. SI-001 overlaps with ruff ANN401.

6. Single bandit version (1.9.4).

7. No ruff baseline verified.

8. TB-029 findings (34) come from .c files in numpy, not from the 39 Python rules.

5. Conclusion

furqan-lint v2.1.0 produces more findings than bandit in 22 of 25 packages (median per-package ratio 5.0x; pooled ratio 0.83x). Two rules account for 58.6% of output. The evidence is suggestive that structural honesty verification occupies different analysis space, but an overlap decomposition and precision measurement are needed to confirm it.

The tool is open-source under Apache-2.0: **https://github.com/BayyinahEnterprise/Kun-Fayakun**

Data availability. benchmark_v2.1.0.csv, raw_json/ (furqan-lint), bandit_json/ (bandit), benchmark.py, test_output_v2.1.0.txt. bandit 1.9.4, Python 3.13.18.

References

[1] B. S. Arfeen, "Structural Honesty Verification: On Surface-Substrate Divergence Across Systems," 2026.

[2] PyCQA, "Bandit: Security linter for Python," https://bandit.readthedocs.io/

[3] M. Brunsfeld et al., "Tree-sitter," https://tree-sitter.github.io/tree-sitter/

[4] OASIS, "SARIF Version 2.1.0," 2020.

بسم الله الرحمن الرحيم